

Minor Project

Malware Detection Tool

Implement basic scanning functionality in your Python-based malware detection tool.

Try to detect malicious files based on hash signatures.

Malware : software designed to harm, exploit, or compromise a system

eg : Virus, Worms, Trojans, Ransomware and Spyware etc.

Malware Detection Tool : software that scans files, applications or systems to detect malicious threats.

It does this by:

- Checking file signatures against a known malware database.
- Monitoring suspicious behavior (e.g., unauthorized access).
- Detecting anomalies in system activity.

Python-Based Malware Detection Tool :

- security script that scans files and directories to identify malicious files based on their **hash signatures**.
- helps to detect known malware by **comparing file hashes** with a database of known malware hashes.

Hash signatures

- unique fixed-length code generated using a cryptographic hashing

algorithm (like SHA-256) from a file's contents.

→ acts like a fingerprint for a file, if the file changes, even slightly, its hash will change completely.

SHA-256

- best choice :
- collision-resistant
- highly secure (256 bit)
- widely trusted
- ideal for cryptographic security and malware detection

Detect malicious files using hash signatures

1. install necessary tools

Python based malware detection tool :

cmd : sudo apt install python3 python3-pip -y

2. set up malware-scanner directory

3. add known malicious SHA-256 hashes in a **malware_hashes.txt** hash database file.

```
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ echo "44d88612fea8a8f36de82e1278abb02f" >> malware_hashes.txt
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ echo "098f6bcd4621d373cade4e832627b4f6" >> malware_hashes.txt
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ echo "5f4dcc3b5aa765d61d8327deb882cf99" >> malware_hashes.txt
```

4. create a **test_malware.txt** file and set its hash to match a hash in **malware_hashes.txt**

```
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ echo "malicious content" > test_malware.txt
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ sha256sum test_malware.txt
cdc374304c02d2967debf17c099ed50100fc5485f0abd991d0c7377561db14cf  test_malware.txt
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ echo "cdc374304c02d2967debf17c099ed50100fc5485f0abd991d0c7377561db14cf" >> malware_hashes.txt
```

5. create the **malware_scanner.py** script

→ Python script script is a basic malware detection tool that scans a directory for malicious files based on their SHA-256 hash signatures.

Functions used in it :

1. load_malware_hashes → This function **reads hashes from malware_hashes.txt** and stores them in a **set** (for fast lookup).

2. calculate_file_hash → calculates the SHA-256 hash of a file.

3. scan_directory → Scans all files in a directory, computes their SHA-256 hash, prints hashes for debugging, and flags files matching known malware hashes.

```
GNU nano 7.2
import os
import hashlib

# Load known malware hashes from file
def load_malware_hashes(file_path):
    try:
        with open(file_path, "r") as f:
            return set(line.strip() for line in f)
    except FileNotFoundError:
        print("[ERROR] Malware hash database not found!")
        return set()

# Calculate SHA-256 hash of a given file
def calculate_file_hash(file_path):
    hasher = hashlib.sha256()
    try:
        with open(file_path, "rb") as f:
            for chunk in iter(lambda: f.read(4096), b''):
                hasher.update(chunk)
            return hasher.hexdigest()
    except Exception as e:
        print(f"[ERROR] Could not read {file_path}: {e}")
        return None

# Scan a directory for malicious files
def scan_directory(directory, malware_hashes):
    print(f"🔍 Scanning directory: {directory}\n")
    suspicious_files = []

    for root, _, files in os.walk(directory):
        for file in files:
            file_path = os.path.join(root, file)
            file_hash = calculate_file_hash(file_path)

            if file_hash:
                print(f"📁 {file}: {file_hash}") # Debug: Print file hashes

                if file_hash and file_hash in malware_hashes:
                    print(f"⚠️ MALICIOUS FILE DETECTED: {file_path}")
                    suspicious_files.append(file_path)

    if not suspicious_files:
        print(f"✅ No malicious files detected.")
    return suspicious_files

if __name__ == "__main__":
    malware_hashes = load_malware_hashes("malware_hashes.txt")
    scan_directory("/home/aditi/malware-scanner", malware_hashes)
    # Change this to your actual username directory
```

6. given a execute permission and run the script

```
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ chmod +x malware_scanner.py
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$ python3 malware_scanner.py
🔍 Scanning directory: /home/aditi/malware-scanner

📁 malware_scanner.py: c87fa4e1ed26452892e83c288f30bb3079ad87998c1d1d76c8bcec518cd02028
📁 malware_hashes.txt: b7fe61ced45153601e057db8700f35fd2198d1bad36eaaee9929ca3379bbf831
📁 test_malware.txt: cdc374304c02d2967debf17c099ed50100fc5485f0abd991d0c7377561db14cf
⚠️ MALICIOUS FILE DETECTED: /home/aditi/malware-scanner/test_malware.txt
aditi@aditi-IdeaPad-3-15IAU7:~/malware-scanner$
```

Result :

Matching against malware database :

- tool compared the hash of each file with the hashes listed in malware_hashes.txt
- the hash of test_malware.txt (cdc374...) matched an entry in malware_hashes.txt

Malicious File Detected :

- Since test_malware.txt matched a known malware hash, the tool flagged it as malicious:

~ feb 15, 2025

~ task 3 week 4